

CLASSES E CONSTRUCTOR EM JAVASCRIPT

`class` • `constructor` • `this` • `new` • **métodos** • **instâncias**

🔗 1. O que é uma Classe?

Uma **classe** é um **molde** usado para criar objetos com características e comportamentos semelhantes.

```
class Pessoa {  
  
}
```

Pense assim:

Classe = molde

Objeto = algo criado a partir do molde

Uma classe pode definir:

- **Propriedades** → dados/características;
- **Métodos** → ações/comportamentos.

◆ 2. `constructor`

O `constructor` é um método especial da classe utilizado para **inicializar os dados de uma nova instância**.

```
class Pessoa {  
  
    constructor(nome, idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
  
}
```

Quando criamos:

```
const pessoa = new Pessoa("Ana", 28);
```

o `constructor` é executado automaticamente.

O resultado é uma instância com:

```
nome → "Ana"  
idade → 28
```

☐ **Para lembrar:**

Constructor = inicializa os dados do objeto.

◆ 3. `this`

`this` representa **a instância atual do objeto**.

```
this.nome = nome;
```

Temos duas coisas diferentes:

```
this.nome
```

↓
propriedade do objeto

nome

↓
parâmetro recebido

Exemplo:

```
const pessoa = new Pessoa("Ana", 28);
```

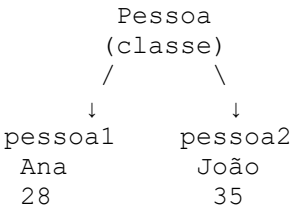
O `this.nome` daquela instância recebe "Ana".

◆ 4. new

O operador `new` é utilizado para **criar uma nova instância da classe**.

```
const pessoa1 = new Pessoa("Ana", 28);  
const pessoa2 = new Pessoa("João", 35);
```

As duas instâncias foram criadas a partir do mesmo molde:



☐ **Para lembrar:**

new = cria uma nova instância.

◆ 5. Métodos

Métodos são **funções definidas dentro da classe** que representam ações que o objeto pode realizar.

```
class Pessoa {  
  
  constructor(nome, idade) {  
    this.nome = nome;  
    this.idade = idade;  
  }  
  
  apresentar() {  
    console.log(`Olá, sou ${this.nome}`);  
  }  
}
```

Podemos executar:

```
const pessoa = new Pessoa("Ana", 28);  
  
pessoa.apresentar();
```

Resultado:

Olá, sou Ana

☐ **Pense:**

Propriedades = características
Métodos = ações

❑ 6. Exemplo completo

```
class Pessoa {  
  
    constructor(nome, idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
  
    envelhecer() {  
        this.idade++;  
    }  
}  
  
const ana = new Pessoa("Ana", 28);  
  
ana.envelhecer();  
  
console.log(ana.idade);
```

Resultado:

29

O método `envelhecer()` modificou a propriedade `idade` da instância.

📦 7. Classe × Instância

Classe	Instância/Objeto
É o molde	É criada a partir do molde
Define propriedades	Possui valores concretos
Define métodos	Executa os métodos
<code>class Pessoa</code>	<code>new Pessoa("Ana", 28)</code>

Exemplo:

```
class Pessoa {  
    constructor(nome) {  
        this.nome = nome;  
    }  
}
```

↑ Classe

```
const pessoa = new Pessoa("Ana");
```

↑ Instância

❑ 8. Aplicação em QA / Playwright

Classes são muito úteis para **organizar testes automatizados**.

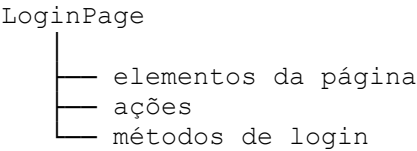
Por exemplo:

```
class LoginPage {  
  
    constructor(page) {  
        this.page = page;  
    }  
  
    async fazerLogin(email, senha) {  
        // ações da página  
    }  
}
```

Depois:

```
const loginPage = new LoginPage(page);
```

A classe pode concentrar:



Isso ajuda a manter o código dos testes **organizado, reutilizável e mais fácil de manter**.

RESUMO PARA A PROVA

class

Cria o molde.

constructor

Inicializa os dados da instância.

this

Representa a instância atual.

new

Cria uma nova instância.

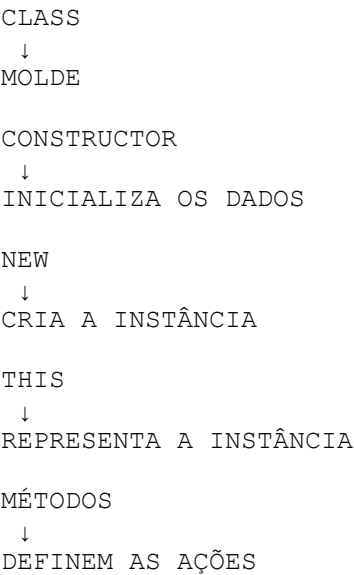
Método

Define uma ação/comportamento.

Propriedade

Representa um dado/característica.

MACETE FINAL



★ **Frase para decorar:**

A classe é o molde; o constructor inicializa os dados; o `new` cria a instância; o `this` representa essa instância; e os métodos definem o que ela pode fazer.

☐ **Exemplo que resume tudo:**

```
class Pessoa {  
  
    constructor(nome) {  
        this.nome = nome;  
    }  
  
    apresentar() {  
        console.log(this.nome);  
    }  
}  
  
const pessoa = new Pessoa("Ana");  
  
pessoa.apresentar();
```

- class** → Pessoa é o molde
- constructor** → recebe nome
- this.nome** → guarda o nome na instância
- new** → cria pessoa
- apresentar()** → executa uma ação